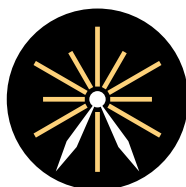


NumPy и pandas

ШПАРГАЛКА



§1 NumPy

Массив `np.ndarray` хранит элементы одного типа. Операцию пишут сразу для всего массива — это векторизация, перебор идёт внутри NumPy, в скомпилированном коде, в десятки раз быстрее цикла по списку. Дробные массивы сравнивают через `np.allclose(a, b)`, а не `==`.

Def (форма и тип).

- `a.ndim` — число осей, `a.shape` — размер по каждой:
(3, 2) — три строки и два столбца, (5,) — одна ось из пяти элементов, а не столбец;
- `a.size` — всего элементов, `a.dtype` — тип: `int64`, `float64`, `bool`. Тип один на массив: целые рядом с дробными тоже станут дробными;
- `a.astype(np.float64)` — сменить тип.

Def (создание).

- `np.zeros(shape)`, `np.ones(shape)`, `np.full(shape, v)`;
- `np.arange(start, stop, step)` — `stop` не входит;
- `np.linspace(start, stop, num)` — `num` точек, оба конца входят; для дробного шага удобнее.

Случайные числа — через генератор с `seed`:

```
rng = np.random.default_rng(42)
rng.integers(1, 81, size=5)      # 81 не входит
rng.random((2, 3))
rng.normal(loc=0, scale=1, size=5)
```

Тот же `seed` и те же вызовы в том же порядке — та же последовательность.

Def (индексы и срезы). Нумерация с нуля, отрицательный индекс — с конца, `a[start:stop:step]`, правая граница не входит. В двумерном `X[i, j]`; двоеточие берёт всю ось: `X[:, :2]` — первые два столбца. `X[[0, 2]]` — строки 0 и 2.

Замечание (срез — не копия). Обычный срез — представление (`view`) тех же данных: изменишь срез — изменится исходный массив. Независимая копия — `.copy()`.

Def (маски). `a >= 2` — булев массив, `a[mask]` оставляет элементы, где `True`. Условия соединяют `&` (и), `|` (или), `~` (не), каждое сравнение в скобках:

```
a[(a < 2) | (a > 4)]
```

Питоновские `and` и `or` здесь не работают.

`np.where(cond, x, y)` — поэлементно `x`, где условие истинно, иначе `y`.

Def (статистики и оси). `sum`, `mean`, `median`, `min`, `max`, `std`, `np.quantile(a, q)` — функцией `np.mean(a)` или методом `a.mean()`.

- `axis=0` сворачивает строки — результат для каждого столбца; `axis=1` — для каждой строки;
- `keepdims=True` оставляет свёрнутую ось размером 1 — удобно для broadcasting;
- `np.std` делит на N , `ddof=0`; на $N - 1$ — `ddof=1`. В `pandas` по умолчанию наоборот;
- `argmin`, `argmax`, `argsort` — индексы минимума, максимума и порядок сортировки.

Def (форма). `a.reshape(3, 4)` — те же элементы в другой форме, их число сохраняется; одну размерность можно задать `-1`, NumPy досчитает. `X.T` — транспонирование. `X[:, None, :]` добавляет ось: (n, d) становится $(n, 1, d)$.

Def (broadcasting). Операция над массивами разной формы. Размеры сравниваются справа налево: в каждой паре они равны или один из них равен 1; недостающие считаются единицами.

1	2		10	20		11	22
3	4	+	10	20	=	13	24
5	6		10	20		15	26
(3, 2)			(2,)			(3, 2)	

- `(3, 2)` и `(2,)` совместимы: `X - X.mean(axis=0)` центрирует столбцы;
- `(3, 2)` и `(3,)` — нет; три числа на три строки — столбцом, `v.reshape(3, 1)`.

Оператор `*` умножает поэлементно, `@` — матрично.

Def (поэлементные и прочие).

- `np.sqrt`, `np.exp`, `np.log`, `np.floor`, `np.round(a, 2)`, `np.clip(a, 0, 1)`;
- `np.sort` — отсортированная копия; `np.unique(a, return_counts=True)` — значения и сколько раз каждое встретилось;
- `np.concatenate([a, b])` склеивает вдоль существующей оси, `np.stack([a, b])` добавляет новую, `axis=1` — столбцами;
- `np.bincount`, `np.add.at` — подсчёты и суммы по целым меткам, накопление по повторяющимся индексам;
- `np.diag`, `np.fill_diagonal` — диагональ.

Замечание (пропуски). Пропуск — `np.nan`. Найти — `np.isnan(a)`, среднее без них — `np.nanmean(a)`. Сравнение `a == np.nan` всегда ложно и для поиска не годится.

Def (линейная алгебра). В `np.linalg`: `norm` — норма, `y` вектора длина; `det`, `matrix_rank`; `solve(A, b)` — решение $Ax = b$, лучше, чем через `inv(A)`; `eigh` — собственные числа и векторы симметричной матрицы; `svd`.

§2 pandas

Def (Series и DataFrame). `pd.Series([1, 7, 2])` — столбец с индексом; `s.iloc[0]` — элемент по позиции; `index=["x", "y", "z"]` — свой индекс. `pd.DataFrame(d)` из словаря `{"a": [...], "b": [...]}` — таблица, по сути словарь столбцов.

Def (чтение и просмотр).

- `pd.read_csv(path)`, `df.to_csv(path, index=False)` — без

лишнего столбца индекса;

- `df.head(10)`, `df.tail(3)`, `df.sample(5)`, `df.columns`, `df.shape`;
- `df.info()` — типы и число непустых значений по столбцам;
- `df.describe()` — статистики числовых столбцов.

Def (доступ).

- `df["Age"]` — столбец, `df[["Age", "Fare"]]` — несколько, `df["new"] = ...` — новый;
- `df.loc[2, "Pclass"]` — по меткам; срез меток *включает* правый конец: `df.loc[2:5, "Survived":"Age"]`;
- `df.iloc[...]` — по позициям, как в NumPy.

Фильтр и запись по условию:

```
df.loc[(df.Sex == "male") & (df.Survived == 0)]
df.loc[df.Survived == 0, ["Survived"]] = -1
```

Def (пропуски). `df.isna()` — маска пропусков,

`df.isna().sum()` — сколько их в каждом столбце.

`df.dropna()` удаляет строки с пропусками, `df.fillna(v)` заполняет. Числовой признак обычно заполняют медианой, категориальный — модой, `s.mode()[0]`. Среднее годится, если распределение симметрично и без выбросов.

Def (агрегации). `sum`, `mean`, `median`, `min`, `max`, `std`

пропускают NaN, а не считают их нулями.

- `.std()` в pandas делит на $N - 1$, `ddof=1`, в NumPy — на N . Сравнивая, задавать `ddof` явно;
- `s.count()` — число непропущенных, `len(s)` — всех строк;
- `s.nunique()` — сколько разных значений;
- `s.value_counts()` — частоты, от частых к редким; `normalize=True` — доли, `dropna=False` — считать и пропуски;
- `s.agg(["min", "max", "mean"])` — несколько статистик сразу.

Def (группировка). `df.groupby("Pclass")["Age"].mean()`

делит строки на группы по значениям столбца и считает

статистику в каждой.

- `as_index=False` — ключи остаются обычными столбцами, а не уходят в индекс;
- `dropna=False` — строки с пропуском в ключе отдельной группой; по умолчанию они выпадают;
- `.transform("mean")` — групповая статистика для каждой исходной строки: стандартизация внутри группы, заполнение пропусков.

Несколько статистик с именами:

```
df.groupby("Pclass", as_index=False).agg(  
    passenger_count=("Age", "size"),  
    known_age_count=("Age", "count"),  
    mean_age=("Age", "mean"),  
)
```

Def (свои функции). `df.apply(f, axis=0)` применяет `f` к каждому столбцу, `axis=1` — к каждой строке; `f` получает `Series`. Быстрее `apply` не делает: что выражается операциями над столбцами, так и пишется — `df["SibSp"] + df["Parch"] + 1`.

Def (ещё).

- `df.drop(columns=[...])`;
- `df.sort_values("col", ascending=False)`;
- `s.idxmax()` — метка первого максимума;
- `df.merge(other, on="key", how="left")` — объединение по ключу; строки левой таблицы остаются все, даже без пары;
- `df.corr()` — корреляции столбцов, `X.corrwith(y)` — каждого с целевой;
- `pd.crosstab(a, b)` — таблица сопряжённости;
- `s.str.extract(regex, expand=False)` — кусок строки по регулярному выражению; `s.replace(..., "Rare")`.

Def (категории). One-hot: столбец на каждое значение, в нём 0 или 1 — `pd.get_dummies(s)` или `OneHotEncoder(sparse_output=False)` из `sklearn`. Label

encoding: каждому значению своё целое число. Проще, но задаёт порядок, которого нет, — Москва, Воронеж и Саратов станут $0 < 1 < 2$, и модель может этим воспользоваться.

Замечание (к домашке). В итоге занятия советуют особенно `sort_values`, `transform`, `merge` и явный `ddof=0`.

§3 Графики

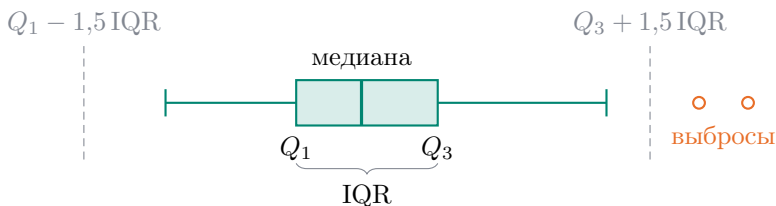
```
plt.figure(figsize=(8, 5))
plt.plot(x, y, color="#68904D", label="y")
plt.legend(); plt.title("..."); plt.xlabel("X")
plt.grid(); plt.show()
```

- `plt.bar`, `plt.barh` — столбцы, `plt.hist(a, bins=20)` — гистограмма;
- `plt.boxplot([a, b])` — боксплоты, `showfliers=False` прячет выбросы;
- `plt.pie(v, autopct="%1.1f%%")` — круговая;
- `plt.axvline(x=m)` — вертикаль, например среднее; `plt.xlim`, `plt.ylim` — пределы осей;
- несколько графиков — `fig, (ax1, ax2) = plt.subplots(1, 2)`, у каждого `ax.set_title(...)`.

Seaborn — тепловая карта, все пары признаков, гистограммы по классам:

```
sns.heatmap(df.corr(), annot=True, fmt=".2f")
sns.pairplot(df)
sns.histplot(data=df, x="Age", hue="Survived",
             kde=True)
```

Def (боксплот). Квартили Q_1 , медиана и Q_3 делят упорядоченные данные на четыре равные части, $IQR = Q_3 - Q_1$. Ящик — от Q_1 до Q_3 , черта в нём — медиана, усы — до крайних точек в пределах $1,5 IQR$ от ящика, дальше — выбросы. Чем шире ящик, тем изменчивее признак.



Замечание (корреляция). Мера линейной зависимости, `df.corr()`. Хорошо, когда признак коррелирует с ответом; плохо, когда признаки сильно коррелируют между собой: это вредит линейным моделям. Тогда оставляют один из признаков, объединяют их в новый или преобразуют нелинейно.

§4 Подготовка к обучению

Замечание (главное правило). Сначала делим на `train` и `test`. Преобразования и модель обучаем на `train`, качество проверяем на `test`. Иначе статистики `test` просочатся в обучение, и оценка выйдет нечестной.

```
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, shuffle=True,
    stratify=y, random_state=42)
```

`shuffle=True` перемешивает строки, `stratify=y` сохраняет доли классов в обеих частях.

Def (масштабирование). `StandardScaler` делает $(x - \mu)/\sigma$, `MinMaxScaler` — $(x - x_{\min})/(x_{\max} - x_{\min})$. Статистики считает `.fit()`, только по `train`; `.transform()` применяет их к любой части:

```
scaler = StandardScaler().fit(X_tr)
X_tr_s = scaler.transform(X_tr)
X_te_s = scaler.transform(X_te)
```

Def (конвейер). Масштаб и модель одним объектом:

```
model = make_pipeline(StandardScaler(),  
                      LinearRegression())  
model.fit(X_tr, y_tr)  
pred = model.predict(X_te)
```

`fit` учит и масштаб, и модель только на `train`, `predict` применяет найденное. Веса —

`model.named_steps["linearregression"].coef_`. После стандартизации вес — изменение ответа при росте признака на одно стандартное отклонение, при прочих равных. Модуль веса не мера важности и не доказывает причинность; при связанных признаках знак веса может не совпасть со знаком корреляции.

Def (оценка). `mean_absolute_error(y_te, pred)`, `r2_score(y_te, pred)`. Ориентир — прогноз средним `train` для всех строк, `np.full(len(y_te), y_tr.mean())`: модель обязана быть лучше.